

1 MultiCASE QSAR Flex Comprehensive Security Document

1.1 Executive Summary

This document provides a comprehensive security analysis of the MultiCASE QSAR Flex ecosystem, encompassing the frontend application, backend services, and user management system. The ecosystem integrates with AWS Cognito for authentication, PostgreSQL for data storage, and includes access to specialized chemical databases. All components are designed with a Kubernetes-first architecture and follow security best practices for enterprise chemical data processing and user management systems.

The QSAR Flex ecosystem consists of three primary components:

1. **QSAR Flex Frontend:** Next.js-based chemical compound management interface
2. **QSAR Flex Backend:** .NET Core-based chemical data processing service
3. **MultiCASE User Manager Backend:** .NET Core-based user management and license verification system

Additional Database Access: The ecosystem includes access to the “Chemical Resources for QSAR Evaluation” database, which can be accessed by services within the Kubernetes cluster for enhanced chemical data processing capabilities.

1.2 Table of Contents

1. [Ecosystem Overview](#)
 2. [Security Architecture](#)
 3. Authentication & Authorization
 4. [Data Security](#)
 5. Input Validation & Sanitization
 6. [File Upload Security](#)
 7. [API Security](#)
 8. [Database Security](#)
 9. [Infrastructure Security](#)
 10. [Third-Party Risk Management](#)
 11. [Chemical Database Access](#)
 12. [Vulnerability Assessment](#)
 13. [Security Recommendations](#)
 14. [Incident Response](#)
 15. [Compliance Considerations](#)
 16. [Security Testing](#)
-

1.3 Ecosystem Overview

The MultiCASE QSAR Flex ecosystem is a comprehensive chemical compound management and user management system designed for enterprise use. The system provides:

1.3.1 Frontend Application (QSAR Flex)

- Chemical compound library management
- Reaction visualization and processing
- User authentication and authorization
- File upload capabilities for chemical data
- Profile management with avatar uploads
- License management for enterprise users

1.3.2 Backend Services

- **QSAR Flex Backend:** Chemical compound data processing and validation
- **User Manager Backend:** User accounts, company structures, and software licenses
- **Chemical Database Service:** Access to “Chemical Resources for QSAR Evaluation” database

1.3.3 Technology Stack

Frontend: - Next.js 15.3.2, React 19, TypeScript - NextAuth.js with AWS Cognito - Radix UI, Tailwind CSS - React Hook Form with Zod validation - Zustand for state management

Backend Services: - .NET Core 8.0 - AWS Cognito integration with JWT validation - PostgreSQL with Entity Framework Core - Docker with Kubernetes deployment - Redis for session management

Infrastructure: - Kubernetes-native deployment - AWS services integration (Cognito, S3, CloudFront) - ECR for container image storage - GitHub Actions for CI/CD pipeline

1.4 Security Architecture

1.4.1 Overall Security Model

The ecosystem follows a secure microservices architecture with the following security layers:

1. **Authentication Layer:** AWS Cognito integration with JWT validation across all services
2. **Authorization Layer:** Role-based access control with company-level isolation
3. **API Security Layer:** JWT token validation and middleware protection
4. **Input Validation Layer:** Comprehensive input validation and sanitization
5. **Data Security Layer:** Secure data handling with encryption and isolation
6. **Database Security Layer:** PostgreSQL with Kubernetes-only access
7. **Chemical Database Layer:** Secure access to specialized chemical databases

1.4.2 Security Boundaries

- **Public Endpoints:** Health checks, authentication endpoints
- **Protected Endpoints:** All application functionality requiring authentication
- **Admin-Only Endpoints:** License creation, company management
- **Database Access:** Restricted to Kubernetes cluster only
- **Chemical Database Access:** Restricted to authenticated services within the cluster

1.4.3 Service Communication Security

Inter-Service Communication:

- All services communicate using JWT tokens created by AWS Cognito
 - Service-to-service communication secured with API tokens for Kubernetes cluster security
 - Network policies restrict inter-service communication
 - TLS encryption for all service communications
-

1.5 Authentication & Authorization

1.5.1 Authentication Implementation

AWS Cognito Integration:

- AWS Cognito creates and issues JWT tokens for all services
- All backend services validate JWT tokens from Cognito
- OAuth 2.0/OpenID Connect implementation
- Automatic token refresh mechanism
- Secure session management across the ecosystem

Security Features:

- JWT token-based authentication (tokens created by AWS Cognito)
- Backend middleware validates Cognito JWT tokens
- Token expiration handling with zero clock skew
- Secure token validation middleware
- Automatic token refresh

Token Management:

- Access tokens with expiration handling
- ID tokens for user identification
- Refresh tokens for seamless re-authentication
- Secure token storage in memory only
- No persistent token storage
- **Single Session Enforcement:** Users can only be logged in from one device at a time
- **Session Invalidation:** New login from another device automatically invalidates previous sessions

- **Security Enhancement:** Prevents session hijacking and unauthorized concurrent access
-

1.5.2 Authorization & Access Control

Role-Based Access Control:

- Company-level isolation (`companyId`) across all services
- Admin privileges (`companyAdmin`) for company management
- Group-based permissions (`cognito:groups`)
- User-level access restrictions
- License-based access control for software modules

Middleware Protection:

- JWT validation middleware on all protected endpoints
- Middleware validates JWT tokens created by AWS Cognito
- Role-based authorization checks
- Company-level data isolation
- Automatic token validation

Protected Endpoint Patterns:

- All application endpoints require valid JWT tokens (created by AWS Cognito)
 - Company-level data isolation enforced across all services
 - Admin-only endpoints with additional privilege checks
 - Service-to-service communication uses API tokens for Kubernetes cluster security
-

1.6 Data Security

1.6.1 Sensitive Data Handling

User Data:

- User credentials stored in AWS Cognito (primary storage)
- User profile information stored in PostgreSQL (secondary copy)
- Company affiliations and admin status
- Authentication tokens (handled by Cognito)

Company Data:

- Company information and structure
- Admin user assignments
- License allocations and usage
- Company-level data isolation

License Data:

- License details and configurations

- User assignments and access rights
- Module selections and coverage
- Test usage tracking and limits

Chemical Data:

- SMILES strings and molecular structures
- Chemical compound information (CAS numbers, names)
- Reaction data and visualization outputs
- **Important:** Chemical data is processed in-memory only and not stored persistently in the main system

Data Encryption:

- HTTPS/TLS encryption in transit
- JWT token encryption
- Database connection encryption
- Secure storage of sensitive configuration

Session Security Controls:

- **Single Session Enforcement:** Users can only be logged in from one device at a time
- **Automatic Session Invalidation:** New login from another device terminates previous sessions
- **Session Hijacking Prevention:** Prevents unauthorized access through session reuse
- **Redis-based Session Management:** Centralized session control with immediate invalidation capabilities

1.6.2 Environment Variable Security

Security Measures:

- Environment variables properly excluded from version control
 - Kubernetes secrets management for sensitive configuration
 - No hardcoded secrets in source code
 - Secure injection via CI/CD pipeline
 - Docker images built and uploaded to ECR by CI/CD
 - Environment variables injected into images during build process
-

1.7 Input Validation & Sanitization

1.7.1 Input Validation

User Data Validation:

- Email format validation
- Company ID validation using GUID format
- Admin status validation

- Input sanitization before processing

Chemical Data Validation:

- SMILES string validation using in-house chemical processing DLL
- InChI format validation with regex patterns
- Chemical structure validation
- Error handling for invalid chemical data

File Validation:

- File extension validation (.smi, .txt, .mol, .sdf, .cdx, .rxn)
- MIME type verification
- File size limitations
- Content validation for chemical files

API Input Validation:

- Request model validation using Data Annotations (.NET)
- Zod schema validation (TypeScript/Next.js)
- Custom validation attributes for business logic
- Comprehensive error messaging
- Input sanitization before processing

1.7.2 XSS Prevention

Content Security:

- All data processed server-side
 - No user input directly rendered as HTML
 - Proper content type validation
 - Secure data handling from trusted sources only
 - Chemical visualization HTML generated server-side from trusted backend sources
-

1.8 File Upload Security

1.8.1 File Type Validation

Chemical File Uploads:

- File extension validation (.smi, .txt, .mol, .sdf, .cdx)
- MIME type verification
- File size limitations
- Content validation for chemical data

Reaction File Uploads:

- File extension validation (.rxn files only)
- Batch file processing support

- Secure file handling

Image Upload Security:

- Image file type validation
- File size and format restrictions
- Secure avatar management
- Presigned URL uploads

Upload Security Measures:

- File type whitelisting
 - MIME type validation
 - File size restrictions
 - Content validation
 - **Important:** Files are processed in-memory only and not stored persistently in the main system
 - Secure file handling with FormData
 - Backend validation and processing
-

1.9 API Security

1.9.1 API Endpoint Protection

Authentication Requirements:

- JWT token validation on all API calls (tokens created by AWS Cognito)
- Backend middleware validates Cognito JWT tokens
- Automatic token refresh handling
- Secure header management
- Error handling with proper status codes

Rate Limiting:

- Request rate limiting per user/IP
- DDoS protection
- API throttling for resource-intensive operations

CORS Configuration:

- Restricted CORS policies
- Allowed origins configuration (multicase.com domains only)
- Secure header handling
- Development policy for local development

Service-to-Service Security:

- API tokens used for service-to-service communication within Kubernetes cluster
- Secure communication between frontend, backend, and user management services

- Network policies restrict inter-service communication
-

1.10 Database Security

1.10.1 PostgreSQL Security

Access Control:

- **Critical Security Feature:** Database is only accessible from within the Kubernetes cluster
- No external network access to database
- Kubernetes service-to-service communication only
- Network policies restrict database access

Connection Security:

- Encrypted database connections
- Connection pooling with security
- Parameterized queries to prevent SQL injection
- No direct database access from external sources

Data Storage:

- User credentials: Primary storage in AWS Cognito, secondary copy in PostgreSQL
- User profiles: Stored in PostgreSQL with company-level isolation
- Company data: Stored in PostgreSQL with access control
- License data: Stored in PostgreSQL with company-level isolation
- Chemical data: **NOT STORED** - processed in-memory only
- File data: **NOT STORED** - processed in-memory only

Backup and Recovery:

- **Regular Database Backups:** PostgreSQL databases are backed up automatically on a regular schedule
- **Point-in-Time Recovery:** Database backups support point-in-time recovery capabilities
- **Geographic Distribution:** Backups are stored in multiple AWS regions for disaster recovery
- **Automated Backup Testing:** Regular verification of backup integrity and recovery procedures

Data Isolation:

- Company-level data separation
 - User-level access restrictions
 - Row-level security considerations
 - Secure data partitioning
-

1.11 Infrastructure Security

1.11.1 Kubernetes Security

Cluster Security:

- RBAC (Role-Based Access Control) enabled
- Network policies restrict inter-service communication
- Pod security policies enforced
- Resource quotas and limits

Service Security:

- Service mesh for secure communication
- TLS termination at ingress
- Secure service-to-service communication
- Network isolation between namespaces
- Enhanced logging and monitoring with Fluentd and OpenSearch
- Alert notifications for security and performance events

Container Security:

- Non-root user execution
- Read-only root filesystem where possible
- Security context restrictions
- Image vulnerability scanning

1.11.2 AWS Integration Security

Cognito Security:

- Password policies and complexity requirements
- Account lockout mechanisms
- Secure token handling
- Multi-factor authentication support

S3 and CloudFront Security:

- Secure file storage with access controls
- CloudFront distribution with security headers
- Presigned URL generation for secure access
- Bucket policies and access controls
- **Automated S3 Backups:** All S3 buckets are automatically backed up with versioning enabled
- **EFS Backup Strategy:** EFS file systems are backed up using AWS Backup service
- **Cross-Region Replication:** Critical data replicated across multiple AWS regions
- **Geographic Redundancy:** CDN distribution provides geographic redundancy and improved performance

Network Security:

- VPC isolation
- Security groups and NACLs
- Private subnets for database access
- VPN access for administrative functions

Amplify Deployment Security:

- Environment variable injection
- Build-time security validation
- Secure artifact handling
- CDN security with CloudFront
- Security alerting and monitoring
- Automated security logging

CloudWatch Monitoring:

- Real-time security monitoring and alerting
 - Comprehensive logging and metrics
 - Automated threat detection
 - Performance and security insights
-

1.12 Third-Party Risk Management

1.12.1 Third-Party Service Providers

AWS Services Risk Assessment:

- **AWS Cognito:** Identity and access management service with SOC 2, PCI DSS, and FedRAMP compliance
- **AWS S3:** Object storage service with 99.999999999% durability and comprehensive security controls
- **AWS ECR:** Container registry with image scanning and vulnerability assessment capabilities
- **AWS CloudFront:** CDN service with DDoS protection and geographic redundancy
- **AWS RDS:** Managed PostgreSQL service with automated backups and encryption

Risk Mitigation Strategies:

- **Service Level Agreements:** All AWS services covered by comprehensive SLAs with availability guarantees
- **Compliance Certifications:** AWS maintains SOC 2, PCI DSS, ISO 27001, and FedRAMP certifications
- **Regular Security Reviews:** Quarterly assessment of third-party service security posture
- **Incident Response Coordination:** Established procedures for coordinating with AWS during security incidents

- **Alternative Service Planning:** Backup plans for critical services in case of AWS service disruption

Data Protection Measures:

- **Encryption in Transit:** All communications with third-party services use TLS 1.2+ encryption
- **Encryption at Rest:** All data stored in third-party services is encrypted using AES-256
- **Access Controls:** Strict IAM policies limit third-party service access to minimum required permissions
- **Audit Logging:** Comprehensive logging of all third-party service interactions
- **Data Residency:** Data stored in AWS regions compliant with organizational data residency requirements

Monitoring and Oversight:

- **Performance Monitoring:** Real-time monitoring of third-party service performance and availability
 - **Security Alerts:** Automated alerts for suspicious third-party service activities
 - **Regular Assessments:** Annual third-party risk assessments and security reviews
 - **Contract Reviews:** Regular review of third-party service agreements and security requirements
-

1.13 Chemical Database Access

1.13.1 “Chemical Resources for QSAR Evaluation” Database

Database Overview:

The ecosystem includes access to the “Chemical Resources for QSAR Evaluation” database, which provides enhanced chemical data processing capabilities for QSAR (Quantitative Structure-Activity Relationship) analysis.

Access Control:

- **Critical Security Feature:** Database access is restricted to authenticated services within the Kubernetes cluster only
- No external network access to the chemical database
- Service-to-service communication secured with API tokens
- Network policies restrict database access to authorized services only

Integration Points:

- QSAR Flex Backend service can access the chemical database for enhanced compound analysis
- User Manager Backend can query chemical database for license validation and access control
- All access is logged and monitored for security compliance

Security Measures:

- Database credentials managed through Kubernetes secrets
- Encrypted connections between services and chemical database
- Access logging and audit trails
- Company-level data isolation maintained
- No persistent storage of chemical database queries in the main system
- **Geographic Redundancy:** Chemical database access distributed across multiple cluster nodes
- **CDN Distribution:** Chemical data delivery optimized through CloudFront for global accessibility

Data Processing:

- Chemical data is retrieved and processed in-memory only
 - No persistent storage of chemical database content in the main PostgreSQL database
 - Secure data handling with proper validation and sanitization
 - Company-level access restrictions enforced
-

1.14 Vulnerability Assessment

1.14.1 Security Strengths

Strong Authentication:

- AWS Cognito integration (creates and issues JWT tokens)
- Backend middleware validates Cognito JWT tokens
- JWT token management across all services
- **Single Session Enforcement:** Prevents concurrent logins and session hijacking
- **Automatic Session Invalidation:** Immediate termination of previous sessions on new login
- Proper session handling
- Role-based access control

Data Security:

- User credentials stored securely in AWS Cognito
- Company-level data isolation across all services
- Secure license management
- Comprehensive audit trail
- No persistent storage of sensitive chemical data

Architecture:

- Clean separation of concerns
- Middleware protection across all services
- Type safety with C# and TypeScript

- Secure routing and service communication

Infrastructure Security:

- Kubernetes-native deployment
 - Network isolation and RBAC
 - Container security best practices
 - AWS security integration
 - Enhanced monitoring and alerting
-

1.15 Security Recommendations

1.15.1 Immediate Actions (Low Priority)

1. Dependency Updates:

- Regular security updates for .NET packages and npm packages
- Automated vulnerability scanning in CI/CD pipeline
- Dependency monitoring and alerting

1.15.2 Medium Priority Actions

1. Enhanced Logging:

- Implement comprehensive security event logging across all services
- Track authentication attempts and failures
- Monitor file upload activities and chemical database access
- Log security-related user actions

2. MFA Implementation:

- Configure multi-factor authentication in AWS Cognito
- Support for SMS, TOTP, or hardware tokens
- Enhanced security for sensitive operations

1.15.3 Long-term Enhancements

1. Compliance Framework:

- SOC 2 Type 2 compliance preparation
 - Data retention policies
 - Privacy impact assessments
-

1.16 Incident Response

1.16.1 Security Incident Classification

Severity Levels:

- **Critical:** Data breach, unauthorized access, system compromise
- **High:** Failed authentication attempts, suspicious license changes, unauthorized chemical database access
- **Medium:** Input validation failures, rate limit violations, file processing issues
- **Low:** Minor configuration issues, dependency vulnerabilities

1.16.2 Response Procedures

1. Immediate Response:

- Isolate affected services
- Preserve evidence
- Notify security team
- Assess scope and impact

2. Containment:

- Block malicious IPs
- Revoke compromised tokens
- Disable affected features
- Update security measures
- Restrict chemical database access if compromised

3. Recovery:

- Restore from secure backups
- Implement additional security measures
- Monitor for recurrence
- Document lessons learned

1.16.3 Communication Plan

- **Internal:** Security team, development team, management
 - **External:** Users (if data compromised), authorities (if required)
 - **Timeline:** Immediate for critical, 24 hours for high, 72 hours for medium
-

1.17 Compliance Considerations

1.17.1 Current Compliance Status

1.17.2 Industry Standards

Security Frameworks:

- OWASP Top 10 compliance
- NIST Cybersecurity Framework
- ISO 27001 considerations

1.17.3 Compliance Preparation

SOC 2 Type 2 Readiness:

- Software architecture designed for SOC 2 Type 2 compliance
- Security controls and processes in place across all services
- No formal audit conducted yet
- Preparation for future compliance requirements

FedRAMP Readiness:

- Software architecture designed for government cloud compliance
 - Security controls aligned with federal requirements
 - Preparation for FedRAMP authorization process
 - Enhanced security controls for government sector
 - No formal FedRAMP audit conducted yet
 - FedRAMP certification planned for near future
-

1.18 Security Testing

1.18.1 Testing Strategy

Automated Testing:

- Static code analysis across all services
- Dependency vulnerability scanning
- Automated security testing
- CI/CD security gates

Manual Testing:

- Penetration testing
- Security code reviews
- Threat modeling
- Risk assessments

1.18.2 Testing Tools

Current Security Testing Tools:

- OWASP ZAP for penetration testing
- Snyk for dependency scanning
- Qodana for code quality and security analysis
- Burp Suite for web application testing
- Kubernetes security scanning tools
- GitHub Actions for automated CI/CD security validation

Testing Frequency:

- Automated: Every build
 - Manual: Quarterly
 - Penetration testing: Annually
 - Security review: Monthly
-

1.19 Conclusion

The MultiCASE QSAR Flex ecosystem demonstrates a strong foundation for security with proper authentication, comprehensive input validation, and secure architecture across all components. The ecosystem’s security posture is significantly enhanced by its Kubernetes-first approach and the critical security decisions implemented across all services.

Key Security Features:

1. **Unified Authentication:** AWS Cognito integration across all services with JWT token validation
2. **User Credentials:** Primary storage in AWS Cognito with secondary copy of user profiles in PostgreSQL (accessible only from Kubernetes cluster)
3. **Chemical Data Processing:** All chemical information is processed in-memory only and not stored persistently in the main system
4. **File Processing:** Uploaded files are processed and analyzed but never stored persistently
5. **Database Security:** PostgreSQL is only accessible from within the Kubernetes cluster, providing strong network isolation
6. **Chemical Database Access:** Secure access to “Chemical Resources for QSAR Evaluation” database restricted to authenticated services within the cluster
7. **CI/CD Security:** Docker images built and uploaded to ECR by CI/CD with secure environment variable injection
8. **Enhanced Monitoring:** Fluentd and OpenSearch integration with alert notifications for comprehensive security monitoring
9. **User Management:** Separate service for user credentials and authentication activities, running on the same cluster with shared authentication
10. **License Verification:** Comprehensive license management with company-level access control and admin-only creation privileges

The ecosystem’s integration with AWS Cognito provides robust authentication capabilities where Cognito creates and issues JWT tokens, while all backend services validate these tokens for user authentication. For service-to-service communication within the Kubernetes cluster, API tokens provide additional security. The use of .NET Core and TypeScript with comprehensive validation ensures type safety and thorough input validation across all services.

The Kubernetes-native architecture provides strong infrastructure security with network isolation, RBAC, and container security best practices. The decision to process chemical data in-memory without persistent storage significantly reduces the attack surface and data exposure risks. The CI/CD pipeline ensures secure image building and deployment to ECR, while

the enhanced logging and monitoring provides comprehensive security visibility and alerting capabilities.

The addition of the “Chemical Resources for QSAR Evaluation” database access enhances the system’s capabilities while maintaining strict security controls. Access is restricted to authenticated services within the cluster, and all interactions are logged and monitored for security compliance.

The software is designed with SOC 2 Type 2 and FedRAMP compliance in mind, though formal audits have not yet been conducted. The architecture includes enhanced security controls suitable for government and enterprise environments. Regular security assessments, monitoring, and updates are essential to maintain security as the ecosystem evolves and new threats emerge.

The current implementation demonstrates a mature understanding of security best practices and provides a solid foundation for continued security enhancements and compliance certifications across the entire QSAR Flex ecosystem.

Document Version: 1.0

Last Updated: August 2025

Next Review: November 2025

Security and Emergency Contact: shrihegde@multicase.com